

12 OCTOBER 2025

POT JS

FUNCTIONS

POT JS function descriptions with examples.

See last pages for complete examples for browser and node. For an introduction and tutorial, see the manual.



ON SYNCHRONOUS CALLS.....	2
INITIALIZATION.....	2
<i>New Map</i>	2
<i>New Map from Saved Reference</i>	3
<i>Save</i>	4
STORING AND RETRIEVING.....	4
<i>Put</i>	4
<i>Get</i>	5
<i>On Loaded</i>	5
RAW BYTE ACCESS.....	6
<i>Store Raw</i>	6
<i>Retrieve from Raw</i>	6
SUPPORT.....	7
<i>Log</i>	7
<i>Hello</i>	7
<i>Optimization</i>	7
<i>Verbosity</i>	8
TESTING.....	9
<i>Random Key</i>	9
<i>Random Value</i>	9
<i>Type-Encoded Bytes</i>	9
<i>Type-Decoded Value</i>	9



SIMULATION TESTING.....	9
<i>Hanging Promise</i>	9
<i>Set Fault</i>	9
<i>Set Panic</i>	10
<i>Set Delay</i>	10
<i>Set Hang</i>	10

ON SYNCHRONOUS CALLS

The core functions – `put()` and `get()` and others – return a promise. There are synchronous variants for most of them, that add 'Sync' to the function name. They can make sense for fast prototyping as they can often be used in synchronous functions. But there are limitations.

Note that synchronous functions do not work with node and not when connecting to a network, as opposed to working with an in-memory POT. The difference is set by using `new()` with or without its optional parameters (with parameters: network, without: in-memory).

INITIALIZATION

NEW KVS

Create a new Swarm map.

`pot.new([<bee url>, <batch id>]) → promise of map`

In-memory (for testing):

```
(async () => {  
    await pot.ready()  
    map = await pot.new()  
    ...  
})()
```

Connecting to a Swarm network. In the example, a local network.

```
(async () => {
```



```
    await pot.ready()

    map = await pot.new("http://localhost:1633",
        "6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa")
    ...
  })()
```

pot.newSync() → map

Synchronous creation of a map (mostly for prototyping):

```
(async () => {

    await pot.ready()

    map = pot.newSync()

    ...
})()
```

NEW MAP FROM SAVED REFERENCE

Create a new Swarm KVS from a reference.

pot.load(reference : int) → promise of map

```
map = await pot.new()

await map.put(key1, val1)

ref = await map.save()

map2 = await pot.load(ref)
```

pot.loadSync(reference : int) → map

Synchronous creation of a map (mostly for prototyping).

The other functions in this example might as well be async.

```
map = pot.newSync()
```



```
map.putSync(key1, val1)

ref = map.saveSync()

map2 = pot.loadSync(ref)
```

SAVE

map.save() → promise of reference

map.saveSync() → reference

Store a Swarm map to the network. Not meaningful for in-memory settings.

Put actions always operate in-memory first, until the entire map is saved to the network.

Examples see, above, **load()**

STORING AND RETRIEVING

The following two are the preferred functions for real-world use.

PUT

Put a value with a code for its type in the 1st byte of the raw value.

get() automatically converts it back to the right type. getRaw() returns all bytes, including the first, undecoded, as Uint8Array.

map.put(key : any, value : any) → promise of null

```
map = await pot.new()

try {
    await map.put("fox", "The quick brown fox jumps!")
} catch(e) {
    ...
}
```

map.putSync(key : any, value : any) → error or null

Store synchronously, with leading type encoding byte. Mostly for prototyping.



GET

Get a value from the POT, interpreting its first byte as type information.

map.get (key : any) → promise of boolean, number or string

```
try {  
    value = await map.get(key)  
} catch(e) {  
    ...  
}
```

When using **.catch**, don't chain it before taking the reference; assign it first.

```
ref = pot.hangingPromise()  
ref.catch((err)=>attestExpectedError(T, err, "Error: canceled"))
```

map.getSync(key : any) → boolean, number or string

Synchronously get the value decoded for its type using the 1st byte of the raw value.

Mostly for prototyping.

ON LOADED

pot.ready() → Promise of null

Delay until the pot object is initialized.

```
await pot.ready()  
  
map = await pot.new()  
  
await map.put("hello", "POT!")
```

onWasmLoaded() → any

This function is called from Go, if it exists, when the initialization of the pot object is done.

```
function onWasmLoaded() {
```



```
    map = pot.newSync()
    map.putSync("hello", "POT!")
    value = map.getSync("hello")
    console.log("key <hello> has:", value)
}
```

RAW BYTE ACCESS

To store `Uint8Arrays` directly, or to inspect the pure byte level beneath the type-encoding, values can be stored as 'raw' bytes without the leading, type encoding byte.

These values can be retrieved raw, by `getRaw()`, or by having them converted immediately to a desired type with `getBoolean()`, `getNumber()`, and `getString()`. Retrieving values stored with `putRaw()` with `get()` or `getSync()` is most of the time not sensible as they would interpret the first byte as type information and then discard it.

STORE RAW

`map.putRaw(key : any, value : Uint8Array) → promise null or Error`

`map.putRawSync(key : any, value : Uint8Array) → null or Error`

Put a raw `Uint8Bytes` array.

RETRIEVE FROM RAW

`map.getRaw(key : any) → Promise of Uint8Array`

`map.getRawSync(key : any) → Uint8Array`

Get a raw `Uint8Bytes` array.

`map.getBoolean(key : any) → Promise of boolean`

`map.getBooleanSync(key : any) → boolean`

Get the raw bytes as Boolean value. First byte 0 being false, all else true.

DO NOT USE ON BOOLEAN VALUE STORED WITH `PUT()` OR `PUTSYNC()`.

`map.getNumber(key : any) → Promise of number`

`map.getNumberSync(key : any) → number`



Get the raw bytes as floating point number (JS default).

DO NOT USE ON NUMBER VALUE STORED WITH PUT() OR PUTSYNC().

map.getString(key : any) → Promise of string

map.getStringSync(key : any) → string

Get the raw bytes as string.

DO NOT USE ON STRING VALUE STORED WITH PUT() OR PUTSYNC().

SUPPORT

LOG

pot.log(message : string)

Write to browser console via Go to verify language-lands roundtrip.

HELLO

pot.hello()

Write hello to browser console to verify established WASM stream.

OPTIMIZATION

To increase robustness where needed, optimization allows to switch off resource releases to achieve a potentially more robust operation at the cost of minor resource leakage.

The default setting is to release resources, the STANDARD level.

0	NONE	resources are not released, the system leaks slightly, might be more robust.
1	STANDARD	resources are released, this is the expectable and default behavior.

There are two ways to set optimization. It can be changed anytime with:

pot.setOptimization(<0 v 1>)

```
(async () => {  
  await pot.ready()  
  pot.setOptimization(pot.NONE) // or 0  
  map = await pot.new()  
})()
```



To set Optimization before the WASM executable is initialized, to suppress even the first two lines it would otherwise log, use `potjs_Optimization`. This works only when it is set before the WASM executable is started. Setting it to a new value after that will not have an effect.

`globalThis.potjs_Optimization = <0 v 1>`

```
var go = new Go()
global.potjs_Optimization = 0 // cannot yet be pot.NONE
WebAssembly.instantiate(
  fs.readFileSync("lib/pot.wasm"), go.importObject)
  .then((r) => { go.run(r.instance) })
```

```
onWasmLoaded = async () => {
```

```
...
```

VERBOSITY

Verbosity means the level of detail of the log to screen or browser console.

The default setting is full DEBUG-level verbosity, which gives the highest amount of log entries for the highest number of details. For production, INFO or less will make sense.

There is no general need to allow errors to make a log entry, as they are also thrown or returned as value by any function call that is affected. NONE can therefore be a valid choice.

Levels

Verbosity can be set to these levels:

0	<code>pot.NONE</code>	no logging, not even errors.
1	<code>pot.CRITICAL</code>	logs only errors that appear to arise from a POT JS malfunction.
2	<code>pot.ERROR</code>	programming and runtime errors are also logged.
3	<code>pot.INFO</code>	general runtime information is logged.
4	<code>pot.DEBUG</code>	specific data, like put and get keys and values are logged.

There are two ways to set the level. Verbosity can be changed anytime with:

`pot.setVerbosity(<LEVEL>)`

Cf. `examples/examples8.js`.

```
(async () => {
  await pot.ready()
  pot.setVerbosity(pot.INFO) // or 3
  map = await pot.new()
})()
```



To set verbosity before the WASM executable is initialized, to suppress even the first two lines it would otherwise log, use `potjs_verbosity`. This works only when it is set before the WASM executable is started. Setting it to a new value after that will not have an effect. The value that matters is the one during WASM initialization.

`globalThis.potjs_verbosity = <0-4>`

See `examples/example7.js`.

```
var go = new Go()
global.potjs_verbosity = 3 // cannot yet be pot.INFO
WebAssembly.instantiate(
  fs.readFileSync("lib/pot.wasm"), go.importObject)
  .then((r) => { go.run(r.instance) })

onWasmLoaded = async () => {
```

... TESTING

These functions exist for black box testing. See `examples/kvs_test.js` for examples of their use.

RANDOM KEY

`pot.randKey()` → `Uint8Array(32)`

32 random bytes for key testing.

RANDOM VALUE

`pot.randValue()` → `Uint8Array(79-101)`

79 to 101 random bytes for value testing (range taken from Go POT).

TYPE-ENCODED BYTES

`pot.type_encoded_bytes(value : any)` → `Uint8Array`

JS type detection and value encoding with correct leading type byte.

TYPE-DECODED VALUE

`pot.type_decoded_value(raw : Uint8Array)` → `boolean`, `number`, or `string`

JS type detection of raw kvs value by leading type byte.



SIMULATION TESTING

The following functions are effective only when the WASM executable has been compiled for the API-side in-memory simulation for extended testing.

HANGING PROMISE

pot.hangingPromise() → promise of null

Blocking promise (in Go) with 1s time out for exception testing.

SET FAULT

pot.setFault()

Make the next function call to the mock storage function return an (JS) error to JS.

SET PANIC

pot.setPanic()

Make the next function call to the mock storage function panic (on the Go side).

SET DELAY

pot.setDelay(msec : int)

Make the next function call to the mock storage function stall for msec milliseconds.

SET HANG

pot.setHang(msec : int)

Make the next function call to the mock storage function stall indefinitely.